# Rift.GG

### 기술 설계 초안 (Technical Specification) · v0.2

프로젝트: LoL 전적 검색 서비스 | 1인 풀스택 포트폴리오 | 초안

개정일: 2026-11-03 | 변경: 백엔드를 Supabase 기반으로 전환

문서 분리 원칙: PRD는 사용자 가치와 기능 범위, 기술 문서는 구현 방법과 아키텍처를 다룹니다.

## 5. 기술 스택

| 영역 | 기술 |
|---|---|
| Frontend | React, TypeScript, Vite |
| Routing | React Router |
| Server State | TanStack Query |
| Client State | Zustand (필요 시) |
| Styling | Tailwind CSS, shadcn/ui (선택) |
| Backend | Supabase Edge Functions (TypeScript / Deno) |
| Database | Supabase PostgreSQL (JSONB 지원) |
| Data Access / Schema | supabase-js, SQL / RPC, Supabase CLI 마이그레이션 |
| Authentication / Authorization | Supabase Auth, PostgreSQL RLS |
| Validation / Chart | Zod, Recharts (선택) |
| Testing | Vitest, React Testing Library, Playwright; Deno 테스트, RLS 검증 |
| Deployment | Vercel (프론트엔드), Supabase (DB / Auth / Edge Functions) |

기존 Node.js / Express API 서버와 Prisma는 Supabase Edge Functions 및 SQL 마이그레이션으로 대체한다. 프론트엔드 개발·빌드에는 Node.js를 사용한다.

## 6. 시스템 아키텍처

### 6.1 데이터 흐름

1. 사용자가 React 프론트엔드에서 Riot ID를 입력한다.
2. React가 supabase-js로 전적 조회 Edge Function을 호출한다.
3. 함수가 입력값과 요청 제한을 확인하고 Supabase PostgreSQL의 캐시를 조회한다.
4. 유효한 캐시가 없으면 함수가 서버 측 Secret의 Riot API 키로 Riot Games API를 호출한다.
5. 응답을 DTO로 변환하고 DB에 저장 또는 갱신한 뒤 JSON을 반환한다.
6. TanStack Query가 응답을 관리하고 React가 화면에 표시한다.

로그인과 세션 관리는 Supabase Auth로 처리한다. 즐겨찾기와 검색 기록은 로그인한 사용자가 supabase-js와 Data API로 접근하며, RLS로 본인 데이터만 허용한다.

### 6.2 설계 원칙

- Riot API는 Edge Functions에서만 호출한다. API 응답 모델과 UI 데이터 모델을 분리한다.
- Riot API 키는 Supabase Secrets에서 관리하며 브라우저에 전달하지 않는다.
- 공용 전적 데이터의 조회·갱신은 Edge Functions를 통한다. 브라우저의 캐시 테이블 직접 접근은 차단한다.
- 사용자 데이터는 Auth와 RLS로 보호한다. 중복 호출 방지와 캐시 TTL을 적용한다.

# 7. 데이터베이스 설계 초안

| 테이블 | 역할 및 접근 정책 |
|---|---|
| auth.users | Supabase Auth가 관리하는 인증 계정 |
| profiles | 기존 Users의 앱 프로필; id가 auth.users.id 참조, 본인만 조회·수정 |
| summoners | 소환사 기본 정보; Edge Functions에서 조회·갱신 |
| rank_entries | 소환사 랭크 정보; Edge Functions에서 조회·갱신 |
| matches | 경기 상세 및 JSONB 원본 일부; Edge Functions에서 조회·갱신 |
| summoner_matches | 소환사-경기 관계; Edge Functions에서 조회·갱신 |
| favorites | 사용자별 즐겨찾기; 본인만 조회·추가·삭제 |
| search_history | 사용자 검색 기록 (추가 기능); 본인만 조회·추가·삭제 |

서비스 사용자 UUID와 Riot 계정 PUUID를 분리한다. favorites와 search_history의 user_id는 auth.users.id를 참조한다. PUUID, match_id 등의 고유 제약과 사용자별 복합 제약으로 중복을 방지한다. 캐시에는 갱신 시각과 만료 시각을 둔다.

스키마·인덱스·RLS 정책·DB 함수는 supabase/migrations의 SQL로 버전 관리하고 CLI로 적용한다. DB 타입을 생성해 TypeScript에서 활용한다. Data API 노출 테이블에는 RLS와 최소 권한을 적용한다. 사용자 테이블 정책은 auth.uid() = user_id (profiles는 id)를 기준으로 SELECT / DELETE의 USING과 INSERT의 WITH CHECK, UPDATE의 양쪽 조건을 검증한다.

# 8. 비기능 기술 요구사항

## 8.1 성능

- DB 캐시 TTL, 페이지 단위 경기 조회, 이미지 지연 로딩으로 요청량을 줄인다.
- 동시 요청은 DB의 원자적 잠금 또는 갱신 상태로 조정한다. 함수 인스턴스 메모리에만 의존하지 않는다.
- 렌더링을 최적화하고 Lighthouse Performance 90점 이상을 목표로 한다.

## 8.2 안정성

- Riot API 장애·호출 제한·네트워크 오류에 대한 일관된 오류 코드를 정의한다.
- 429와 Retry-After를 반영하고 재시도 횟수를 제한한다. 중복 저장은 upsert와 고유 제약으로 방지한다.
- Edge Functions 실행 제한 안에서 조회 범위를 제한하고 타임아웃을 설정한다. 대량 수집은 별도 작업 설계가 필요하다.

## 8.3 보안

- 브라우저에는 Supabase URL과 publishable key만 둔다. Riot 키와 secret / service_role 키는 서버 전용이다.
- 개인 데이터 요청은 유효한 Auth 사용자와 RLS 정책을 확인한다. 관리자 권한은 RLS를 우회하므로 캐시 처리에 한정한다.
- 비로그인 전적 검색은 공개 함수로 설계하고, JWT 필수 여부를 명시적으로 설정한다. 공개 호출에도 입력 검증과 공유 저장소 기반 요청 제한을 적용한다.
- Zod 입력 검증, 허용 출처 CORS, 로그의 토큰·비밀정보 제거를 적용한다. CORS를 인증 수단으로 사용하지 않는다.

## 9. 구현 범위 및 후속 결정 사항

- 프론트엔드: React, TypeScript, Vite, React Router, TanStack Query, Tailwind CSS 중심으로 시작한다.
- 백엔드: Edge Functions가 Riot API 호출, DTO 변환, 입력 검증, 캐시 갱신 및 요청 제한을 담당한다.
- 데이터: Supabase PostgreSQL과 supabase-js를 사용한다. 트랜잭션이 필요한 작업은 DB 함수 / RPC로 처리하고 SQL 마이그레이션으로 관리한다.
- 인증: Supabase Auth로 로그인·세션을 관리하고, 개인 데이터 접근은 RLS로 제한한다.
- 배포: 프론트엔드는 Vercel, DB·Auth·함수는 Supabase에 배포한다. 개발·운영 환경과 Secrets를 분리한다.
- 검증: 프론트엔드 테스트 외에 함수의 캐시 적중·미적중·Riot 오류와 익명·본인·타인의 RLS 접근을 검증한다.
- Zustand, shadcn/ui, Recharts는 실제 UI 요구가 발생할 때 도입 여부를 확정한다.
- 후속 상세 설계: ERD, 함수 API 명세, TTL, 오류 코드, 요청 제한 저장소·기준, 공개 함수 인증 설정, 테스트 전략, 운영 플랜·리전을 확정한다.

## 10. 외부 의존성 및 정책

Riot API 개발 키와 공개 서비스용 승인 요건을 구분한다. 운영 전 최신 Riot 개발자 정책을 확인한다. Supabase 운영 플랜의 실행 시간·호출량·DB 용량과 비용도 배포 전에 확인한다.

공식 참고 문서 (확인일: 2026-11-03)

Riot API | https://developer.riotgames.com/apis

Riot 개발자 포털 | https://developer.riotgames.com/docs/portal

Supabase Edge Functions | https://supabase.com/docs/guides/functions

Supabase Row Level Security | https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase Secrets | https://supabase.com/docs/guides/functions/secrets

Supabase SQL 마이그레이션 | https://supabase.com/docs/guides/deployment/database-migrations

Supabase Edge Functions 제한 | https://supabase.com/docs/guides/functions/limits

개정 기록: v0.1의 백엔드·DB 접근·인증·배포 설명을 Supabase에 맞춰 갱신하고, 중복된 장 번호를 8~10으로 정리했다. 기능 범위와 프론트엔드 기술 선택은 유지했다.